TECHNOLOGIA
Former. Performer. Transformer.

FORMATION IA132 · 2 JOURS

Guide des ateliers pratiques

IA pour développeurs : code assisté et programmation intelligente

LE DÉPÔT DES LABS

github.com/housseméddinelassoued/ia132-labs

Houssem Eddine Lassoued
Formateur · Technologia

12 labs indépendants
Python et TypeScript

AVANT DE COMMENCER

Mode d'emploi

Chaque lab est **indépendant** : un dossier, un objectif, un point de départ et des tests qui disent quand c'est terminé. Vous pouvez en reprendre un à tout moment, dans l'ordre que vous voulez, après la formation.

1 · RÉCUPÉRER LE DÉPÔT

1. Ouvrez `https://github.com/houssemeddinelassoued/ia132-labs` et cliquez sur **Fork** : vous obtenez votre propre copie, où vous pourrez pousser et ouvrir des pull requests.
2. Clonez **votre** fork, puis vérifiez votre poste :

```
git clone https://github.com/<votre-compte>/ia132-labs.git
cd ia132-labs
python scripts/verifier_env.py
```

3. **Python** : `python -m venv .venv`, activez-le, puis `pip install -r requirements-dev.txt`. **TypeScript** : rien à installer, Node.js 22.18+ exécute les fichiers `.ts` directement.
4. Faites le **lab 00** : deux tests verts et votre assistant qui « voit » le dépôt.

2 · LE CYCLE DE CHAQUE LAB

Rouge → vert

Au départ, la plupart des tests échouent : c'est l'exercice. Le lab est terminé quand ils passent, **sans modifier les tests** (sauf consigne).

```
pytest labs/lab04-debogage      # Python
npm run test:lab04              # TypeScript
```

L'IA propose, vous décidez

Relisez chaque suggestion et chaque diff. Lancez les tests après chaque changement. Un commit par étape réussie : `git restore .` annule une étape ratée.

Classe virtuelle

Chacun travaille sur son poste, minuteur à l'écran. Points d'étape réguliers : répondez au micro « étape 1, 2 ou 3 ». Bloqué ? Dites-le : partage d'écran de 2 minutes.

Confidentialité

Aucun code ni donnée de votre entreprise dans ces exercices. Les projets et les données des labs sont fictifs.

3 · LIRE UNE FICHE

Objectif	La compétence visée, en une phrase.
Point de départ	Les fichiers fournis et l'état des tests au départ.
Étapes	Le déroulé conseillé, avec les demandes (prompts) à adapter.
Livrable	Les critères objectifs de fin : cochez-les.
Pièges	Ce que l'IA fait souvent mal sur ce type de tâche : à surveiller.
Bonus	Pour aller plus loin si vous avez fini en avance.

VUE D'ENSEMBLE

Les 12 labs et les modules

Les horaires sont indicatifs. Les labs 00 à 05 se font le jour 1, les labs 06 à 11 le jour 2.

LAB	TITRE ET OBJECTIF	MODULE	MOMENT	DURÉE
00	**Mise en route** Avoir un poste prêt pour les 2 jours : le dépôt cloné, les tests qui passent, l'assistant d'IA actif dans votre IDE.	Ouverture	Jour 1 9 h 20	10 min
01	**Les 4 modes de votre assistant** Utiliser les 4 modes d'interaction d'un assistant de codage — complétion en ligne, chat, édition, agent — et savoir lequel choisir selon la tâche.	1 · Panorama des assistants de codage	Jour 1 10 h 05	20 min
02	**Générer du code à partir d'une spécification** Rédiger un prompt structuré (contexte, tâche, contraintes, format, exemples) et utiliser des tests existants comme critère d'acceptation du code généré.	2 · Génération et réécriture de code	Jour 1 11 h 25	25 min
03	**Refactoriser du code hérité, sans rien casser** Refactoriser un module hérité par petites étapes vérifiées : l'IA propose, les tests de caractérisation protègent, Git garde chaque étape.	2 · Génération et réécriture de code	Jour 1 13 h 00	25 min
04	**Chasse aux bogues avec l'IA** Déboguer méthodiquement avec un assistant : reproduire, faire expliquer, formuler des hypothèses, corriger, puis prouver la correction par un test de non-régression.	3 · Débogage et optimisation avec l'IA	Jour 1 13 h 55	25 min
05	**Optimiser avec l'IA… et le prouver par la mesure** Faire analyser la complexité d'un code par l'IA, appliquer l'optimisation proposée, puis mesurer le gain sans changer le comportement.	3 · Débogage et optimisation avec l'IA	Jour 1 14 h 55	20 min
06	**Documenter automatiquement… et vérifier ce qui est écrit** Générer une documentation conforme à une convention (docstrings Google / TSDoc), un README de module et un journal des modifications, puis vérifier que la documentation dit vrai.	4 · Revue de code et documentation automatisée	Jour 2 9 h 25	20 min
07	**Revue de code assistée par l'IA** Mener une revue de pull request avec l'IA et un analyseur statique, classer les constats par gravité, trier les faux positifs, puis corriger les problèmes bloquants.	4 · Revue de code et documentation automatisée	Jour 2 10 h 00	25 min
08	**Générer des tests unitaires qui servent vraiment** Faire générer une suite de tests par l'IA (cas nominaux, limites, erreurs), puis la juger avec deux mesures objectives : la couverture des branches et le score de mutation.	5 · Génération de tests et assurance qualité	Jour 2 11 h 05	30 min
09	**Tests d'intégration, charge et pipeline CI/CD** Faire écrire par l'IA des tests d'intégration sur une vraie API, simuler une charge avec un objectif de service, puis brancher le tout dans un pipeline GitHub Actions qui bloque une PR fautive.	5 · Génération de tests et assurance qualité	Jour 2 11 h 40	35 min

LAB	TITRE ET OBJECTIF	MODULE	MOMENT	DURÉE
10	**Concevoir et planifier avec l'IA** Utiliser l'IA comme partenaire d'architecture : faire émerger les questions, comparer des options, documenter une décision (ADR), découper et estimer un backlog réaliste, puis auditer des dépendances… en vérifiant chaque affirmation.	6 · IA pour la planification et l'architecture logicielle	Jour 2 13 h 40	35 min
11	**Projet final : un outil complet, de la spécification à la PR** Développer de bout en bout un petit outil avec un agent de codage — plan, code, tests, documentation, CI, pull request revue — en appliquant le flux de travail d'une équipe : instructions de projet, petits commits, revue humaine.	7 · Ateliers pratiques	Jour 2 14 h 30	55 min

LAB
00

OUVERTURE

Mise en route

⏱ 10 min | Jour 1 · vers 9 h 20 | Python et/ou TypeScript | `labs/lab00-mise-en-route/`

OBJECTIF

Avoir un poste prêt pour les 2 jours : le dépôt cloné, les tests qui passent, l'assistant d'IA actif dans votre IDE.

POINT DE DÉPART

ÉTAPES

1 **Forkez** le dépôt sur votre compte GitHub (bouton *Fork*), puis clonez **votre** fork :

```
git clone https://github.com/<votre-compte>/ia132-labs.git
cd ia132-labs
```

2 **Vérifiez vos outils** :

```
python scripts/verifier_env.py
```

3 **Python** — créez un environnement virtuel et installez les outils :

```
python -m venv .venv
# Windows : .venv\Scripts\activate   ·   macOS / Linux : source .venv/bin/activate
pip install -r requirements-dev.txt
pytest labs/lab00-mise-en-route
```

4 **TypeScript** — rien à installer, Node 22.18+ exécute les fichiers `.ts` :

```
npm run test:lab00
```

5 **Assistant d'IA** — ouvrez le chat de votre outil (Copilot, Cursor, JetBrains AI, Claude Code…) et demandez :

> *Résume ce dépôt en 5 lignes, puis liste les labs avec leur objectif.*

Vérifiez que la réponse cite bien les dossiers `labs/…` : l'assistant « voit » votre projet.

6 Notez le **nom du modèle** utilisé par votre assistant (sélecteur de modèle du chat).

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] `pytest labs/lab00-mise-en-route` et/ou `npm run test:lab00` : **2 tests verts**.
- [ ] Votre assistant répond dans le chat en citant les fichiers du dépôt.

PIÈGES À OBSERVER

COINCÉ ?

- `python` introuvable sous Windows : essayez `py` .
- Proxy d'entreprise : `pip install --proxy http://proxy:port -r requirements-dev.txt` .
- Pas de Copilot ? Un essai gratuit suffit ; sinon Cursor ou Claude Code font l'affaire pour tous les labs.

MES NOTES

LAB 01

1 · PANORAMA DES ASSISTANTS DE CODAGE

Les 4 modes de votre assistant

⏱ 20 min | Jour 1 · vers 10 h 05 | Python ou TypeScript | `labs/lab01-decouverte/`

OBJECTIF

Utiliser les **4 modes d'interaction** d'un assistant de codage — complétion en ligne, chat, édition, agent — et savoir lequel choisir selon la tâche.

POINT DE DÉPART

- `python/convertisseur.py` ou `typescript/convertisseur.ts` : 5 fonctions avec leur signature et leur documentation, mais sans code.
- Les tests des étapes 1 à 3 (`test_convertisseur.py` / `convertisseur.test.ts`) et de l'étape 4 (`test_cli.py` / `cli.test.ts`) échouent tous au départ.

```
pytest labs/lab01-decouverte          # Python
npm run test:lab01                    # TypeScript
```

ÉTAPES

1 **Complétion en ligne (5 min)** — Effacez la ligne `raise NotImplementedError` (ou `throw`) de `celsius_vers_fahrenheit`, placez le curseur dans la fonction et attendez la suggestion grisée. `Tab` pour accepter, `Échap` pour refuser. Faites de même pour les 3 conversions suivantes. **Lisez chaque suggestion avant de l'accepter.**

2 **Chat (5 min)** — Sélectionnez `fahrenheit_vers_celsius` et demandez :

> *Explique cette fonction. Quels cas limites ne sont pas couverts par les tests ?*

Notez une idée de cas limite pertinente… et une qui ne l'est pas.

3 **Édition (5 min)** — Ouvrez le mode édition (Copilot Edits, Cursor `Ctrl+K`, JetBrains *AI Actions*) et demandez :

> *Implémente convertir() avec un dictionnaire de fonctions plutôt qu'une suite de if. Lève ValueError pour une paire non prise en charge.*

Relisez le **diff** proposé avant de l'accepter. Lancez les tests : étapes 1 à 3 au vert.

4 **Agent (5 min)** — Passez en mode agent (Copilot *Agent*, Cursor *Agent*, Claude Code, Junie) et demandez :

> *Ajoute une fonction main(argv) qui rend le module utilisable en ligne de commande, de façon à faire passer les tests de test_cli. Lance les tests pour vérifier.*

Observez : quels fichiers l'agent lit-il ? Quelles commandes lance-t-il ? Vous demande-t-il votre accord ?

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] Tous les tests du lab au vert.
- [ ] Un fichier `NOTES.md` dans le dossier du lab : pour chaque mode, **une phrase** « utile pour… / à éviter pour… ».

PIÈGES À OBSERVER

- La complétion invente parfois une constante au lieu d'utiliser `KM_PAR_MILE` déjà définie.
- En mode agent, l'outil peut modifier les **tests** pour les faire passer : refusez, et dites-le dans votre demande.

BONUS

Demandez au chat la même implémentation avec **deux modèles différents** (sélecteur de modèle) et comparez : longueur, style, cas limites.

MES NOTES

LAB 02

2 · GÉNÉRATION ET RÉÉCRITURE DE CODE

Générer du code à partir d'une spécification

⏱ 25 min | Jour 1 · vers 11 h 25 | Python ou TypeScript | `labs/lab02-generation/`

OBJECTIF

Rédiger un **prompt structuré** (contexte, tâche, contraintes, format, exemples) et utiliser des **tests existants comme critère d'acceptation** du code généré.

POINT DE DÉPART

- `SPEC.md` : les règles métier (TPS, TVQ, rabais, livraison, arrondi).
- `python/facture.py` ou `typescript/facture.ts` : la signature seule.
- `test_facture.py` / `facture.test.ts` : 12 tests d'acceptation, tous en échec. **Ne les modifiez pas.**

ÉTAPES

1. **Lisez** `SPEC.md` (3 min). Repérez les 2 règles les plus piégeuses.
2. **Rédigez votre prompt** dans un fichier `PROMPT.md` en suivant le canevas :

```
CONTEXTE    : langage, version, fichier, conventions du projet
TÂCHE       : ce que la fonction doit faire (renvoyer à SPEC.md avec #file / @SPEC.md)
CONTRAINTES : entiers en cents, arrondi demi vers le haut, pas de dépendance, exceptions…
FORMAT      : une seule fonction + fonctions privées, sans modifier les tests
EXEMPLES    : 1 ou 2 cas chiffrés tirés de SPEC.md
```

3. **Générez** l'implémentation (chat ou édition), en joignant `SPEC.md` et le fichier de tests au contexte.
4. **Lancez les tests**. S'il en reste en échec : copiez **le message d'échec** dans le chat plutôt que de demander « corrige ».
5. **Relisez** le code : chaque règle de `SPEC.md` est-elle visible et nommée (constantes, petites fonctions) ?

```
pytest labs/lab02-generation       # Python
npm run test:lab02                 # TypeScript
```

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] 12 tests au vert, sans modification des tests.
- [ ] `PROMPT.md` : votre prompt final + le nombre d'itérations nécessaires.

PIÈGES À OBSERVER

- `round()` en Python arrondit 50,5 à 50 : le test d'arrondi le détecte.
- Des montants en `float` (`0.05 * base`) : refusez-les, la spécification impose des entiers.
- L'assistant qui « corrige » un test pour le faire passer.

BONUS

Demandez à l'assistant 5 cas limites **absents** des tests (ex. code `ETE15` à exactement 100,00 $). Ajoutez ceux qui sont justes… après avoir vérifié le calcul à la main.

MES NOTES

LAB 03

2 · GÉNÉRATION ET RÉÉCRITURE DE CODE

Refactoriser du code hérité, sans rien casser

⏱ 25 min | Jour 1 · vers 13 h 00 | Python ou TypeScript | `labs/lab03-refactorisation/`

OBJECTIF

Refactoriser un module hérité **par petites étapes vérifiées** : l'IA propose, les tests de caractérisation protègent, Git garde chaque étape.

POINT DE DÉPART

- `python/commandes.py` ou `typescript/commandes.ts` : une fonction `traiter(c, r, p)` de 50 lignes — noms d'une lettre, `if` imbriqués sur 4 niveaux, nombres magiques, code dupliqué par région.
- `test_commandes.py` / `commandes.test.ts` : **tests de caractérisation**, verts au départ. Ils figent le comportement actuel (fichier `donnees/attendu.json`).

```
pytest labs/lab03-refactorisation                          # Python
ruff check --select C901 labs/lab03-refactorisation        # complexité (Python)
npm run test:lab03                                         # TypeScript
```

ÉTAPES

1. **Diagnostic (5 min)** — Demandez au chat :

   > *Liste les problèmes de lisibilité et de maintenabilité de traiter(), du plus coûteux au moins coûteux. Ne propose pas encore de code.*

   Notez la complexité de départ (`ruff check --select C901` signale `traiter` comme trop complexe).

2. **Plan (3 min)** — Demandez un **plan de refactorisation en 5 étapes maximum**, chacune ne changeant pas le comportement. Validez ou réordonnez le plan vous-même.

3. **Exécution pas à pas (15 min)** — Pour chaque étape : demandez **cette étape seulement** → relisez le diff → lancez les tests → `git commit -m "refactor: …"`. Si un test casse : `git restore .` et reformulez. Pistes : noms explicites, constantes nommées, grille de tarifs par région (dictionnaire / dataclass), fonctions `frais_livraison`, `priorite`, retours anticipés (`continue`) au lieu des `if` imbriqués, types.

4. **Bilan (2 min)** — Relancez `ruff check --select C901` : `traiter` ne doit plus apparaître.

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] Tests de caractérisation **toujours verts**, sans les avoir modifiés.
- [ ] 3 à 5 commits `refactor: …`, chacun vert.
- [ ] `traiter` sous le seuil de complexité (8).

PIÈGES À OBSERVER

- « J'en ai profité pour corriger un bogue » : en refactorisation, **aucun** changement de comportement. Les arrondis (`round(t + f, 2)`) et les seuils stricts (`> 100`) doivent rester identiques.
- L'IA remplace le `print` par une exception : c'est un changement de comportement. Un `logging.warning` est acceptable ; discutez-en.
- Une grosse réécriture d'un seul coup : impossible à relire, impossible à annuler proprement.

BONUS

Demandez à l'assistant d'ajouter des **types** (`TypedDict` / `dataclass` en Python, `type Commande` en TypeScript) et vérifiez que les tests passent encore.

MES NOTES

LAB 04

3 · DÉBOGAGE ET OPTIMISATION AVEC L'IA

Chasse aux bogues avec l'IA

⏱ 25 min | Jour 1 · vers 13 h 55 | Python ou TypeScript | `labs/lab04-debogage/`

OBJECTIF

Déboguer **méthodiquement** avec un assistant : reproduire, faire expliquer, formuler des hypothèses, corriger, puis **prouver** la correction par un test de non-régression.

POINT DE DÉPART

- `rapport-incidents.md` : 5 tickets du support (symptômes, une trace d'erreur).
- `python/reservations.py` ou `typescript/reservations.ts` : le module en production.
- `test_reservations.py` / `reservations.test.ts` : 4 tests verts, 5 tests rouges (un par ticket).

```
pytest labs/lab04-debogage            # Python
npm run test:lab04                    # TypeScript
```

ÉTAPES

1. **Reproduire INC-101 (3 min)** — Lancez `python tableau_de_bord.py` (ou la version `.ts`). Vous obtenez la trace du rapport.

2. **Faire expliquer, pas corriger (5 min)** — Collez la trace **complète** dans le chat avec le fichier en contexte :

   > *Voici une trace d'erreur et le code concerné. Explique la cause racine en 3 phrases, puis propose 2 corrections possibles avec leurs inconvénients. N'écris pas encore le code.*

   Choisissez vous-même la correction, puis appliquez-la.

3. **Un ticket à la fois (12 min)** — Pour INC-102 à INC-104 : donnez au chat **le symptôme + le test rouge**, demandez une hypothèse, vérifiez-la (en lisant le code ou avec `print` / débogueur), corrigez, relancez **tous** les tests.

4. **Non-régression (5 min)** — Pour chaque correction, vérifiez qu'un test échoue si vous annulez la correction (`git stash` → test rouge → `git stash pop`).

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] 9 tests au vert (INC-105 en bonus).
- [ ] Un commit par ticket : `fix(INC-10x): …`, avec la cause racine en une ligne dans le message.

PIÈGES À OBSERVER

- L'IA propose `try/except ZeroDivisionError: return 0` partout : ça masque le symptôme au lieu de traiter la cause (pas de réservation → pas de jour).
- INC-104 : `round(x + 0.0001, 2)` « marche » sur ce cas… et échoue sur d'autres. Exigez une explication (représentation binaire des décimaux) et une correction de fond (`Decimal` , calcul en cents).
- Un test modifié pour devenir vert n'est pas une correction.

BONUS

INC-105 : faites trouver la cause par l'IA **sans** lui montrer le test, seulement le ticket. A-t-elle vu le piège de `.seconds` / `getHours()` ?

MES NOTES

LAB 05

3 · DÉBOGAGE ET OPTIMISATION AVEC L'IA

Optimiser avec l'IA… et le prouver par la mesure

⏱ 20 min | Jour 1 · vers 14 h 55 | Python ou TypeScript | `labs/lab05-performance/`

OBJECTIF

Faire analyser la **complexité** d'un code par l'IA, appliquer l'optimisation proposée, puis **mesurer** le gain sans changer le comportement.

POINT DE DÉPART

- `analyse.py` / `analyse.ts` : 3 fonctions correctes mais quadratiques (`in` sur une liste, `count` dans une boucle).
- `test_analyse.py` / `analyse.test.ts` : le comportement à préserver (verts).
- `test_performance.py` / fin de `analyse.test.ts` : 2 budgets de temps (rouges).
- `bench.py` / `bench.ts` : chronomètre les fonctions quand la taille double.

```
python labs/lab05-performance/python/bench.py     # ou : npm run bench:lab05
pytest labs/lab05-performance                     # ou : npm run test:lab05
```

ÉTAPES

1. **Mesurer d'abord (3 min)** — Lancez le banc d'essai. Quand la taille double, le temps est-il ×2 ou ×4 ? Notez les chiffres.
2. **Faire analyser (5 min)** — Demandez :

   > *Donne la complexité en temps de chaque fonction de ce fichier, en expliquant la ligne responsable. Propose une version O(n) qui garde exactement le même résultat, y compris l'ordre des éléments.*

3. **Appliquer et vérifier (7 min)** — Appliquez une fonction à la fois, relancez **les tests de comportement** après chacune.
4. **Mesurer après (5 min)** — Relancez le banc d'essai, puis `bench.py 200000`. Les budgets de temps passent-ils ?

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] Tous les tests au vert, dont les 2 budgets de temps.
- [ ] Un tableau avant / après dans `NOTES.md` (3 tailles × 3 fonctions).

PIÈGES À OBSERVER

- `list(set(courriels))` est rapide… mais **perd l'ordre** : un test le détecte.
- `Counter.most_common()` départage les égalités par ordre d'apparition, pas alphabétique.
- « C'est plus rapide » sans mesure n'est pas une preuve. Une micro-optimisation (compréhension de liste au lieu d'une boucle) ne change pas un $O(n^2)$.

BONUS

Demandez à l'IA un profil d'exécution (`python -m cProfile -s cumtime bench.py`, ou `node --cpu-prof`) et faites-lui interpréter le résultat.

MES NOTES

LAB 06

4 · REVUE DE CODE ET DOCUMENTATION AUTOMATISÉE

Documenter automatiquement… et vérifier ce qui est écrit

⏱ 20 min | Jour 2 · vers 9 h 25 | Python ou TypeScript | `labs/lab06-documentation/`

OBJECTIF

Générer une documentation **conforme à une convention** (docstrings Google / TSDoc), un README de module et un journal des modifications, puis **vérifier** que la documentation dit vrai.

POINT DE DÉPART

- `geo.py` / `geo.ts` : 4 fonctions géographiques, **zéro** commentaire.
- Les tests de comportement sont verts ; les tests de documentation sont rouges : ils exigent `Args:` / `Returns:` / `Raises:` et un exemple exécutable (Python, vérifié par `doctest`), ou `@param` / `@returns` / `@throws` / `@example` (TypeScript).

```
pytest labs/lab06-documentation              # ou : npm run test:lab06
python -m doctest -v labs/lab06-documentation/python/geo.py
```

ÉTAPES

1. **Convention d'abord (3 min)** — Demandez à l'assistant de documenter **une** fonction en précisant la convention (« docstring au format Google, en français, avec un exemple doctest exécutable »). Comparez avec le même prompt **sans** préciser la convention.
2. **Tout le module (7 min)** — Documentez le module, la classe `Point` et les 4 fonctions (commande `/doc` de Copilot, *Generate documentation* de JetBrains, ou le chat). Lancez les tests.
3. **Vérifier (5 min)** — Les exemples sont-ils justes ? Python : `doctest` le prouve. TypeScript : exécutez un exemple à la main. Cherchez une phrase **fausse** ou invérifiable (ex. « précision au mètre »).
4. **README et changelog (5 min)** — Demandez un `README.md` pour ce module (but, installation, exemple, limites), puis :

   > *À partir de `git log --oneline -15`, rédige une entrée de CHANGELOG au format Keep a Changelog (Ajouté / Modifié / Corrigé).*

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] Tous les tests du lab au vert.
- [ ] `README.md` du module, revu par vous (au moins une correction de votre part).

PIÈGES À OBSERVER

- L'IA paraphrase le code (« incrémente i de 1 ») au lieu d'expliquer le **pourquoi** (haversine, Terre sphérique, bord inclus).
- Les exemples inventés : `distance_km(Montréal, Québec) == 250` « à peu près » → le doctest échoue.
- La documentation qui promet un comportement que le code n'a pas (ex. « lève une erreur si la liste est vide »).

BONUS

Générez la doc HTML : `python -m pydoc -w geo` ou `npx typedoc labs/lab06-documentation/typescript/geo.ts`.

MES NOTES

LAB 07

4 · REVUE DE CODE ET DOCUMENTATION AUTOMATISÉE

Revue de code assistée par l'IA

⏱ 25 min | Jour 2 · vers 10 h 00 | Python ou TypeScript | `labs/lab07-revue/`

OBJECTIF

Mener une **revue de pull request** avec l'IA et un analyseur statique, **classer** les constats par gravité, trier les faux positifs, puis corriger les problèmes bloquants.

POINT DE DÉPART

- `notes_service.py` / `notes-service.ts` : la PR n° 42 « Service de notes, prêt pour la prod ». Les tests fonctionnels passent.
- `grille-revue.md` : la grille de gravité et le gabarit de commentaire.
- `.github/copilot-instructions.md` (racine du dépôt) : les conventions de l'équipe, lues par Copilot.

ÉTAPES

1. **Revue humaine éclair (3 min)** — Lisez le fichier **sans IA** et notez ce qui vous frappe.
2. **Revue par l'IA (7 min)** — Demandez :

   > *Fais la revue de ce fichier comme un relecteur senior, en suivant grille-revue.md. Pour chaque constat : ligne, gravité (Bloquant / Majeur / Mineur / Suggestion), problème, correction proposée. Termine par un verdict : approuver ou demander des changements.*

   Puis l'analyseur statique :

   ```
   ruff check labs/lab07-revue/python                # règles S (bandit) : SQL, MD5, secret en dur…
   ```

3. **Trier (5 min)** — Comparez les trois listes (vous, l'IA, `ruff`). Marquez chaque constat de l'IA **vrai / faux positif / discutable**. Puis lancez les tests de sécurité :

   ```
   pytest labs/lab07-revue                           # ou : npm run test:lab07
   ```

   Votre revue avait-elle vu les 5 problèmes que les tests révèlent ?

4. **Corriger les bloquants (10 min)** — Requêtes paramétrées, empreinte de mot de passe salée et lente (`hashlib.pbkdf2_hmac`, `scrypt`), contrôle du propriétaire, secret lu dans l'environnement. Un commit par correction.

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] `REVUE.md` : vos constats classés (tableau), avec la colonne « trouvé par : moi / IA / ruff ».
- [ ] Les 9 tests au vert.

PIÈGES À OBSERVER

- L'IA signale 15 « problèmes », dont la moitié de style : un relecteur trie, il ne recopie pas.
- Elle peut manquer le **contrôle d'accès** (`lire_note` ne vérifie pas le propriétaire) : aucun outil statique ne le voit, c'est une règle métier.
- `except Exception: return None` cache les erreurs (et ici l'injection SQL).

BONUS

Sur votre fork, ouvrez une vraie PR avec ce fichier et demandez une revue à Copilot (*Reviewers → Copilot*). Comparez avec votre `REVUE.md` .

MES NOTES

LAB
08

5 · GÉNÉRATION DE TESTS ET ASSURANCE QUALITÉ

Générer des tests unitaires qui servent vraiment

⏱ 30 min | Jour 2 · vers 11 h 05 | Python ou TypeScript | `labs/lab08-tests/`

OBJECTIF

Faire générer une suite de tests par l'IA (cas nominaux, limites, erreurs), puis la juger avec deux **mesures objectives** : la couverture des branches et le **score de mutation**.

POINT DE DÉPART

- `panier.py` / `panier.ts` : un panier d'achat (ajout, retrait, codes promo, livraison) sans tests.
- `test_panier.py` / `panier.test.ts` : 2 tests de départ.
- `mutants.py` / `mutants.ts` : introduit 8 bogues un par un et vérifie que vos tests les détectent.

```
# Python
pytest labs/lab08-tests --cov=panier --cov-branch --cov-report=term-missing
python labs/lab08-tests/python/mutants.py
# TypeScript
npm run cov:lab08
node labs/lab08-tests/typescript/mutants.ts
```

Au départ : couverture ≈ 40 %, score de mutation **1/8**.

ÉTAPES

1. **Lister les comportements d'abord (5 min)** — Demandez :

   > *Liste les comportements de cette classe à tester : cas nominaux, valeurs limites, erreurs. Un comportement par ligne, sans code.*

   Ajoutez-en un que l'IA a oublié.

2. **Générer (10 min)** — Faites générer les tests (`/tests` dans Copilot, *Generate tests* dans JetBrains, ou le chat), **avec** : fixtures, `pytest.mark.parametrize` / boucles de `test`, noms de tests en phrases.

3. **Mesurer (5 min)** — Couverture des branches, puis mutants. Pour chaque mutant **survivant**, demandez à l'IA le test qui le tuerait.

4. **Relire (5 min)** — Chaque test vérifie-t-il quelque chose ? Supprimez ceux qui ne font qu'exécuter du code (`assert panier is not None`).

5. **Valeurs limites (5 min)** — Vérifiez qu'il existe un test **exactement au seuil** (99 articles, 50,00 $ de livraison gratuite, stock exact).

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] Couverture des branches **≥ 95 %**.
- [ ] Score de mutation **8/8**.
- [ ] Tous les tests au vert, en moins d'une seconde.

PIÈGES À OBSERVER

- 100 % de couverture et 3/8 mutants tués : la couverture mesure ce qui **s'exécute**, pas ce qui est **vérifié**.
- L'IA calcule l'attendu avec la même formule que le code (`assert total == sous_total - sous_total // 10`) : le test recopie le bogue. Écrivez des valeurs littérales (`9_998 - 999`).
- Les tests qui dépendent les uns des autres (panier partagé, sans fixture).

BONUS

Demandez des tests **par propriétés** (`hypothesis` en Python, `fast-check` en TypeScript) : « le total n'est jamais négatif », « ajouter puis retirer laisse le panier inchangé ».

MES NOTES

LAB 09

5 · GÉNÉRATION DE TESTS ET ASSURANCE QUALITÉ

Tests d'intégration, charge et pipeline CI/CD

⏱ 35 min | Jour 2 · vers 11 h 40 | Python et/ou TypeScript + GitHub Actions | `labs/lab09-ci/`

OBJECTIF

Faire écrire par l'IA des **tests d'intégration** sur une vraie API, simuler une **charge** avec un objectif de service, puis brancher le tout dans un **pipeline GitHub Actions** qui bloque une PR fautive.

POINT DE DÉPART

- `api.py` / `api.ts` : une mini-API de liens courts (création, redirection, statistiques), sans dépendance.
- Tests unitaires existants (verts) ; `test_integration.py` / `integration.test.ts` : un seul test (`/sante`).
- `charge.py` / `charge.ts` : test de charge (80 % de lectures) avec objectif p95 < 50 ms et < 1 % d'erreurs.
- `ci-a-completer.yml` : un pipeline à trous ; `python/test_workflow.py` : ses critères d'acceptation (rouges).

```
pytest labs/lab09-ci                                  # ou : npm run test:lab09
python labs/lab09-ci/python/charge.py                 # ou : node labs/lab09-ci/typescript/charge.ts
```

ÉTAPES

1. **Scénarios d'intégration (10 min)** — Demandez :

   > *Voici une API HTTP (api.py) et la fixture de test qui démarre un vrai serveur. Liste les scénarios d'intégration à couvrir (nominal, erreurs 4xx, concurrence), puis écris-les en réutilisant requete().*

   Visez au moins 6 scénarios, dont **200 créations concurrentes** qui doivent donner 200 codes uniques.

2. **Couverture (3 min)** — `pytest labs/lab09-ci/python --cov=api --cov-report=term-missing` : ≥ 80 % ?

3. **Charge (7 min)** — Lancez `charge.py`, puis `charge.py 5000 100`. Faites **interpréter** les chiffres à l'IA (p50 vs p99, débit, erreurs). Demandez-lui un second scénario « pic » (100 % de créations) et comparez.

4. **Pipeline (12 min)** — Copiez `ci-a-completer.yml` dans `.github/workflows/ci.yml` et faites compléter les 6 TODO par l'IA. Validez localement :

   ```
   pytest labs/lab09-ci/python/test_workflow.py
   ```

5. **Le voir bloquer (3 min)** — Poussez sur votre fork, ouvrez une PR qui casse un test (ex. `LONGUEUR_CODE = 5`) : le pipeline doit passer **au rouge**. Fermez la PR.

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] Tests d'intégration verts, couverture ≥ 80 %.
- [ ] `.github/workflows/ci.yml` : critères de `test_workflow.py` au vert, exécution verte sur GitHub (onglet *Actions*).
- [ ] Une phrase dans `NOTES.md` : « l'objectif de service est / n'est pas respecté parce que… ».

PIÈGES À OBSERVER

- Des tests d'intégration qui appellent les fonctions en mémoire (`Depot()`) : ce sont des tests unitaires déguisés.
- Un port fixe (8009) dans les tests : échecs aléatoires en CI. Utilisez le port 0.
- L'IA invente des actions (`actions/setup-python@v9`) ou des options : vérifiez sur la page de l'action.
- Des secrets écrits dans le YAML : utilisez `${{ secrets.NOM }}` .

BONUS

Ajoutez au pipeline une étape qui lance `charge.py 1000 10` et échoue si l'objectif de service n'est pas respecté.

MES NOTES

LAB 10

6 · IA POUR LA PLANIFICATION ET L'ARCHITECTURE LOGICIELLE

Concevoir et planifier avec l'IA

⏱ 35 min | Jour 2 · vers 13 h 40 | Markdown, Mermaid, CSV (vérification en Python ou TypeScript) | `labs/lab10-architecture/`

OBJECTIF

Utiliser l'IA comme **partenaire d'architecture** : faire émerger les questions, comparer des options, documenter une décision (ADR), découper et estimer un backlog réaliste, puis auditer des dépendances… en **vérifiant** chaque affirmation.

POINT DE DÉPART

- `cahier-des-charges.md` : Cantine+, précommande de repas pour 3 cafétérias (projet fictif).
- `gabarits/` : modèle d'ADR et en-tête du backlog CSV.
- `dependances/` : les fichiers de dépendances d'un vieux prototype que l'équipe veut réutiliser.
- `python/test_livrables.py` / `typescript/livrables.test.ts` : vérifient que vos livrables sont **complets et cohérents** (pas leur qualité).

Créez vos fichiers dans `labs/lab10-architecture/livrables/`.

```
pytest labs/lab10-architecture          # ou : npm run test:lab10
```

ÉTAPES

1. **Questions d'abord (5 min)** — Joignez le cahier des charges et demandez :

   > *Avant de proposer une architecture, pose-moi les 8 questions les plus importantes sur ce projet, classées par impact sur l'architecture.*

   Répondez à 3 d'entre elles (vous jouez le PO).

2. **Architecture (10 min)** — Demandez **2 options contrastées** (ex. monolithe modulaire vs microservices) avec leurs compromis face aux contraintes (pic de 10 h 30, paie SAP, allergènes, équipe de 4). Choisissez. Faites produire `livrables/architecture.md` avec un diagramme **Mermaid** et un tableau des composants.

3. **ADR (5 min)** — `livrables/adr-001.md` à partir du gabarit : votre décision, ses alternatives, ses conséquences.

4. **Backlog et planification (12 min)** — Faites découper le projet en **12 user stories ou plus** (`livrables/backlog.csv`) : critère d'acceptation Gherkin, points (suite de Fibonacci), priorité MoSCoW, sprint, dépendances. Lancez les tests : l'IA respecte-t-elle la **vélocité de 20 points**, l'ordre des dépendances, le pilote en 3 sprints ? Corrigez.

5. **Risques et dépendances (8 min)** — `livrables/risques.md` (≥ 5 risques : probabilité, impact, mesure). Puis faites analyser `dependances/` par l'IA et **vérifiez** avec un outil :

   ```
   pip install pip-audit && pip-audit -r labs/lab10-architecture/dependances/requirements-legacy.txt
   cd labs/lab10-architecture/dependances && npm audit --package-lock-only   # après npm install --package-lock-only
   ```

   Consignez le résultat dans `livrables/dependances.md`, en signalant chaque identifiant CVE inventé par l'IA.

LIVRABLE · C'EST TERMINÉ QUAND…

Les 5 fichiers de `livrables/`, tous les tests du lab au vert, et une phrase dans `NOTES.md` : « l'IA a été utile pour… / je l'ai corrigée sur… ».

PIÈGES À OBSERVER

- L'IA propose des microservices, Kubernetes et Kafka pour 4 développeurs et un pilote en 6 semaines.
- Les estimations « au doigt mouillé » : demandez-lui de justifier chaque 8 par un critère (inconnue, intégration, risque).
- Sprints surchargés et dépendances inversées : c'est exactement ce que les tests détectent.
- Des numéros de CVE plausibles… mais inexistants. Seul un outil d'audit fait foi.

BONUS

Demandez à l'IA de jouer l'**avocat du diable** sur votre ADR : quel scénario ferait regretter cette décision dans 2 ans ?

MES NOTES

LAB 11

7 · ATELIERS PRATIQUES

Projet final : un outil complet, de la spécification à la PR

⏱ 55 min | Jour 2 · vers 14 h 30 | Python ou TypeScript | `labs/lab11-projet-final/`

OBJECTIF

Développer **de bout en bout** un petit outil avec un agent de codage — plan, code, tests, documentation, CI, pull request revue — en appliquant le flux de travail d'une équipe : instructions de projet, petits commits, revue humaine.

POINT DE DÉPART

- `SPEC.md` : `taches`, un gestionnaire de tâches en ligne de commande (6 commandes, règles, codes de sortie).
- `test_acceptation.py` / `acceptation.test.ts` : 16 (Python) ou 11 (TypeScript) tests d'acceptation, tous rouges. **Ne les modifiez pas.**
- `taches/cli.py` / `taches/cli.ts` : la signature de `main()`, rien d'autre.
- `AGENTS.md` : un gabarit d'instructions pour l'agent, à compléter.

```
pytest labs/lab11-projet-final          # ou : npm run test:lab11
cd labs/lab11-projet-final/python && python -m taches lister     # ou : npm run taches -- lister
```

ÉTAPES

1. **Préparer l'agent (5 min)** — Créez une branche `lab11`. Complétez `AGENTS.md` avec l'IA : conventions, commandes de test, ce qu'il ne doit pas faire. C'est ce fichier qui rend le travail reproductible pour toute une équipe.

2. **Plan avant le code (5 min)** — En mode agent ou plan :

   > *Lis SPEC.md, AGENTS.md et les tests d'acceptation. Propose un plan d'implémentation en étapes, avec les fichiers à créer. N'écris pas de code tant que je n'ai pas validé le plan.*

   Corrigez le plan (découpage `modele` / `depot` / `cli`), puis validez.

3. **Implémenter par incréments (25 min)** — Une commande à la fois : `ajouter` + `lister`, puis `terminer` / `supprimer`, puis `en-retard`, puis `exporter`. Après chaque incrément : tests verts → relecture du diff → commit `feat: …`. Ajoutez au moins **5 tests unitaires à vous** (modèle, ordre de tri, dates invalides).

4. **Documenter (5 min)** — Faites générer un `README.md` d'utilisation (installation, exemples de chaque commande, codes de sortie) et vérifiez chaque exemple en le lançant.

5. **Pull request (10 min)** — Poussez la branche, ouvrez une PR vers `main` de **votre fork**. Description générée par l'IA, puis **relue** par vous. Demandez une revue IA (Copilot *Reviewers*, ou collez le diff dans le chat avec `grille-revue.md` du lab 07). Traitez les commentaires utiles, répondez aux autres.

6. **Rétrospective (5 min)** — Dans `NOTES.md` : temps passé, ce que l'agent a bien fait, 2 erreurs qu'il a commises et comment vous les avez détectées.

LIVRABLE · C'EST TERMINÉ QUAND…

- [ ] Tous les tests d'acceptation au vert + vos tests unitaires.
- [ ] `README.md`, `AGENTS.md`, `NOTES.md` complétés.
- [ ] Une PR sur votre fork, avec la revue et vos réponses.
- [ ] **Présentation : partage d'écran de 2 à 3 minutes** (démo de l'outil + une leçon apprise).

PIÈGES À OBSERVER

- L'agent écrit tout d'un coup, 400 lignes, sans plan : arrêtez-le et revenez à l'étape 2.
- `argparse` qui appelle `sys.exit()` sur une erreur : la spécification exige un **code retourné** (2).
- Les identifiants réutilisés après suppression : un test le vérifie.
- Une PR « générée et fusionnée » sans relecture : c'est vous qui en répondez.

BONUS

- Ajoutez la commande `modifier <id> [--titre] [--priorite] [--echeance]`, avec ses tests d'acceptation écrits **avant** le code.
- Ajoutez le job du lab 11 au pipeline CI du lab 09.

MES NOTES

ANNEXE A

Canevas de demande (prompt) pour le code

Cinq ingrédients. Plus la tâche est risquée, plus chacun doit être précis.

```
CONTEXTE    Langage, version, framework, fichier(s) concerné(s), conventions (#file, @workspace)
TÂCHE       Ce qu'il faut faire, en une phrase, avec le verbe d'action (implémente, refactorise,
explique…)
CONTRAINTES Ce qui ne doit PAS changer : comportement, API publique, tests, dépendances,
performance
FORMAT      Ce que vous attendez : un diff, une fonction, un plan sans code, un tableau, des tests
EXEMPLES    1 ou 2 cas concrets : entrée → sortie attendue, ou un extrait du style voulu
```

Demandes qui marchent

- « Pose-moi tes questions avant d'écrire le code. »
- « Propose un plan en 5 étapes ; n'écris pas encore de code. »
- « Voici le message d'échec du test : explique la cause racine. »
- « Donne 2 options avec leurs inconvénients ; je choisis. »
- « Ne modifie pas les tests. Lance-les après chaque changement. »

Demandes à éviter

- « Corrige. » (sans trace ni attendu)
- « Améliore ce code. » (améliorer selon quel critère ?)
- « Écris toute l'application. » (impossible à relire)
- Coller un secret, une clé d'API ou des données personnelles.
- Accepter sans lancer les tests.

CHOISIR LE MODE D'INTERACTION

Complétion	Code répétitif, prévisible, local : une ligne ou une fonction dont l'intention est claire (nom, type, docstring).
Chat	Comprendre, expliquer, comparer des options, déboguer à partir d'une trace. Aucune modification sans votre accord.
Édition	Modifier un ou plusieurs fichiers que vous désignez, avec un diff à relire.
Agent	Tâche en plusieurs étapes : lire le dépôt, modifier, lancer les tests, itérer. À encadrer par un plan et des instructions de projet.

VALIDER AVANT D'ACCEPTER

- [ ] Je comprends chaque ligne, et je saurais l'expliquer en revue.
- [ ] Les tests passent, et un test échoue si je casse le code.
- [ ] Les API, options et paquets cités existent vraiment (documentation officielle).
- [ ] Pas de faille évidente : entrées non fiables, secrets, SQL, chemins, cryptographie.
- [ ] Le diff ne touche que ce que j'ai demandé.

ANNEXE B

Aide-mémoire des commandes

Python

```
pytest labs/lab02-generation
pytest labs/lab08-tests --cov=panier --
cov-branch \
       --cov-report=term-missing
ruff check labs/lab07-revue
ruff check --select C901 labs/lab03-
refactorisation
python -m doctest -v labs/lab06-
documentation/python/geo.py
python labs/lab05-
performance/python/bench.py
python -m cProfile -s cumtime bench.py
```

TypeScript (Node.js 22.18+)

```
npm run test:lab02
npm run cov:lab08
node labs/lab08-tests/typescript/mutants.ts
npm run bench:lab05
npm run api:lab09
npm run taches -- lister
npm install && npm run typecheck   #
facultatif
```

Git

```
git switch -c lab04              # une
branche par lab
git add -p                       #
relire avant d'indexer
git commit -m "fix(INC-101): …"
git restore .                    #
annuler une étape ratée
git stash / git stash pop        #
prouver un test de non-régression
git log --oneline -15
```

Fichiers d'instructions

Copilot	`.github/copilot-instructions.md`
Agents (standard ouvert)	`AGENTS.md`
Cursor	`.cursor/rules/*.mdc`
Claude Code	`CLAUDE.md`
JetBrains	`.junie/guidelines.md`

MON PLAN D'ACTION : 3 USAGES QUE J'ADOPTE DÈS LUNDI